SauceDemo – SQA Practical QA & AI Assessment

Application: https://www.saucedemo.com/ · **Primary account:** `standard_user` · **Tested:** 6–7 Oct 2026 **Environment:** Ubuntu Linux 6.14 · **Google Chrome 146.0.7680.164** (primary) · Firefox 155.0 (cross-browser check) · Playwright 1.63.0 · Desktop 1280×720 and mobile 375×667

---

1. Summary

- I explored about 25 risk hypotheses across login, catalogue, cart, checkout, menu, session handling and navigation. Every one was run in a real browser through scripted Playwright probes (`explore/*.probe.js`), so each one can be re-run.
- I found **5 defects on `standard_user`**. The two most serious are in checkout: an **empty cart can be "ordered"**, and **the customer-information step can be skipped or filled with blanks**.
- I **rejected or found not reproducible 11 hypotheses**. Some were plainly wrong (missing form labels, broken sorting, tax maths). Others were the intended behaviour of a backend-less demo, or bugs planted in the other test accounts.
- **Automation:** a Playwright + JavaScript suite with page objects, running on Google Chrome by default. The core suite has 1 end-to-end purchase test and 4 negative tests. All pass on Chrome in about 3 seconds, and on Chrome plus Firefox together when cross-browser mode is switched on. I also proved it can fail: changing the expected tax rate produced a clear, correctly located failure. All 18 documented test cases are automated in three separate suites: core (5 tests, pass), regression (18 tests, pass), and known defects (6 tests that fail until D1–D5 are fixed).

---

2. Exploratory testing & AI prompt log (Task A)

2.1 Risk ranking (impact × likelihood)

#	Area	Why it matters	Risk
1	Checkout flow integrity (order creation, totals)	Revenue, chargebacks, fulfilment of bad orders	**High**
2	Checkout data validation & step order	Undeliverable orders, data quality	**High**
3	Cart state across sessions/users	Privacy on shared devices, wrong orders	**Med-High**
4	Authentication & route protection	Unauthorised access	Medium (client-side demo)

#	Area	Why it matters	Risk
5	Cart ↔ UI state consistency (add/remove/reset)	Wrong purchases, user confusion	Medium
6	Catalogue correctness (sorting, prices list vs detail)	Trust, wrong price shown	Medium
7	Accessibility & responsive layout	Excludes users, legal exposure	Medium
8	URL tampering (invalid IDs)	Crashes, corrupt state	Low-Med

2.2 Hypotheses tested

C = confirmed, **NR** = not reproducible, **R** = rejected.

ID	Hypothesis (AI-generated unless noted)	Result	Notes
H01	Checkout possible with empty cart	**C → D1**	$0.00 order reaches "Thank you for your order!"
H02	Deep-link to `/checkout-step-two.html` skips customer info	**C → D2**	Finish works; `/checkout-complete.html` also directly reachable
H07	Whitespace-only / nonsense postal / 500-char / HTML in fields	**C → D2** (whitespace)	HTML is not executed (no XSS), so I counted only the whitespace case as a defect
H05/H15	Cart persists after logout and shows for next user	**C → D3**	`localStorage.cart-contents` not user-scoped, not cleared on logout
H06	Reset App State leaves UI out of sync	**C → D4**	Badge cleared, "Remove" buttons remain until reload
H14	Invalid product id can be added to cart	**C → D5** (my own follow-up)	The AI only suggested "invalid id shows an error page", which is the intended easter egg. I took it further and added the item to the cart
H16	Back button after Finish returns to checkout	C, folded into D1	Lands on a $0.00 overview with an active Finish button
H03	Protected pages reachable without login	**NR**	All 5 routes redirect to login with a clear error
H04a	Back button after logout shows inventory	**NR**	Redirected to login with an error
H04b	Session is a forgeable client cookie (`session-username`, not HttpOnly)	**R – by design**	Backend-less demo: auth is simulated client-side. I'd raise it on a real product but not here
H08	Subtotal/tax/total mis-calculated	**NR**	3 items: 95.97 + 7.68 (8 %) = 103.65 ✔

ID	Hypothesis (AI-generated unless noted)	Result	Notes
H09	Sorting incorrect / not retained	**NR**	All 4 sort options correct. Resetting to A-Z after navigating away is a reasonable default
H10	Inputs lack labels / images lack alt / not keyboard-operable	**R – AI was wrong**	Inputs have aria-labels, images have alt text, Tab reaches Add-to-cart
H11	Mobile layout overflows horizontally	**NR**	No horizontal overflow at 375 px
H12	Login is case/whitespace-insensitive, unclear messages	**NR**	Exact-match usernames, distinct messages, password masked
—	Cart quantity cannot be changed	**R – unsupported assumption**	No quantity control exists, so it is a product decision rather than a defect
—	Cancel on overview goes to Products, not Cart	**R – by design / UX opinion**	Recorded as a UX note only
—	`problem_user` identical images & broken sort, `error_user` Finish fails, `performance_glitch_user` ~5 s login, `visual_user` misplaced Checkout button	**R – seeded**	Planted bugs on purpose-built accounts, used only for comparison and **not reported**

2.3 Representative prompts

P1 – Risk map

> *"I'm testing saucedemo.com (a demo e-commerce site with no real backend), logged in as standard_user. Rank the top 8 user-journey risks by user impact, business impact and likelihood. For each, give 2 concrete exploratory tests I can run in under 2 minutes. Do not assume features that aren't visible in the UI."*

- **Useful output:** the risk table in 2.1 and the H01–H13 hypotheses.
- **What I validated or changed:** I ranked "auth/route protection" only #4. There is no backend, so a client-side cookie can't be a real security boundary on this app, and H04b confirmed it is simulated. Of the AI's accessibility, sorting, maths and mobile ideas, all came back **not reproducible or wrong** (H08–H11) and were dropped.

P2 – Checkout edge cases

> *"For SauceDemo's checkout (Your Information → Overview → Complete), list edge cases covering input validation, direct URL access to later steps, empty cart, refresh/back-button behaviour, and price/tax calculation. Flag which are likely 'by design' for a demo app."*

- **Useful output:** the H01, H02, H07 and H16 ideas, all four of which became defects D1 and D2.

- **What I rejected:** "postal code must be numeric", because no locale is specified and many countries use alphanumeric codes. I also rejected "XSS via first name". I tested it and the field is rendered as text, never executed, so I discarded it.

P3 – Classify a finding (used for every candidate defect)

> *"Here is what I observed: <steps + actual>. Is this a genuine defect, a limitation of a demo app, or intended behaviour of the test account? Argue both sides, then give severity and priority with reasoning."*

- **Useful output:** it made me check each finding against the seeded accounts and the "no backend" constraint.
- **What I classified as not a defect:** the forgeable `session-username` cookie (H04b) looks alarming but is "by design" for a demo with no server-side session. The misplaced Checkout button seen while gathering D3 evidence came from the `visual_user` session, and that account's visual bugs are planted on purpose, so I excluded it.
- **What I took further:** the "ITEM NOT FOUND / $√-1" page is a deliberate easter egg. Instead of stopping there, I added that item to the cart, which exposed the cart corruption and checkout crash in D5.

P4 – Generate the automation

> *"Write a Playwright JavaScript test using the Page Object Model for: login → add 2 named products → verify the cart → checkout → verify item total, tax and total → finish → verify confirmation. Use data-test selectors, web-first assertions, no fixed waits."*

The output was a working scaffold. Section 4 lists what was wrong with it.

P5 – Adversarial review

> *"Review this test as a strict senior SDET. Find brittle selectors, weak or missing assertions, hidden waits, isolation problems, and anything that could pass falsely."*

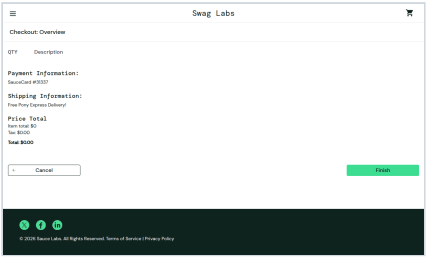
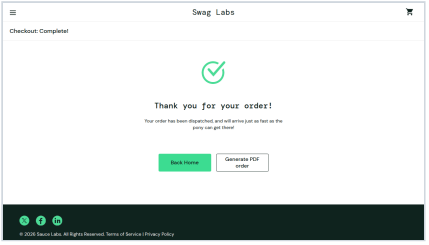
This review found the unguarded read in the overview step (C2 in section 4). It also prompted the "prove it can fail" mutation check.

---

3. Defect report (Task B)

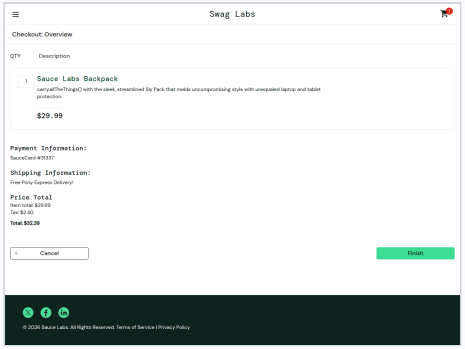
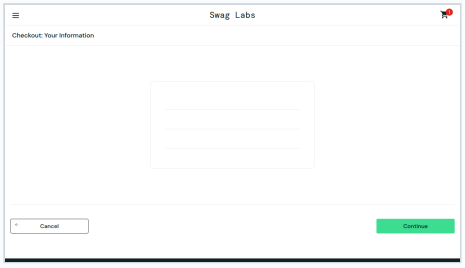
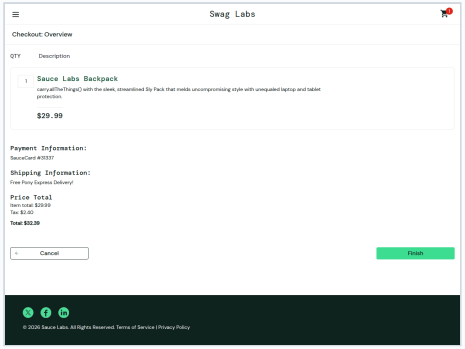
All defects reproduce on **`standard_user`** in Google Chrome 146. I first found them in Playwright's bundled Chromium and Firefox, then re-ran every probe in Chrome, and the screenshots below are from Chrome. They are ordered by risk.

D1 – Order can be placed with an empty cart ($0.00 order confirmed)

Severity / Priority	High / P1
Steps	1. Log in as `standard_user`. 2. Without adding anything, click the cart icon → **Checkout**. 3. Enter any First/Last/Postal → **Continue**. 4. Overview shows `Total: $0.00` → click **Finish**.
Expected	Checkout is disabled or blocked for an empty cart with a message ("Your cart is empty").
Actual	The flow completes and shows "Thank you for your order! Your order has been dispatched…". After that, browser **Back** returns to an active $0.00 overview where Finish can be clicked again.
Impact	Creates empty or duplicate orders, which pollutes order and fulfilment data and analytics. Users are told something was "dispatched" when nothing was bought.
Why a defect, not by design	Every e-commerce checkout guards against empty orders. The app already shows a "dispatched" confirmation, so it treats the order as real, and nothing in the UI suggests $0 orders are intended.
Evidence	 , 

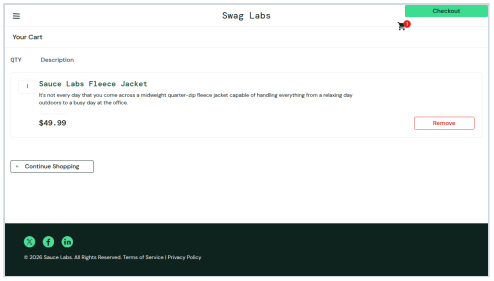
D2 – Customer-information step can be bypassed or filled with blanks

Severity / Priority	High / P2
Steps (a) – step skip	1. Log in, add *Sauce Labs Backpack*. 2. Type `https://www.saucedemo.com/checkout-step-two.html` in the address bar. 3. Click **Finish**.
Steps (b) – blank data	1. Log in, add an item, cart → **Checkout**. 2. Enter three spaces in First Name, Last Name and Postal Code → **Continue**.
Expected	(a) Redirect to "Your Information" until it is valid. (b) Values that are empty after trimming are rejected with the same "is required" error shown for truly empty fields.

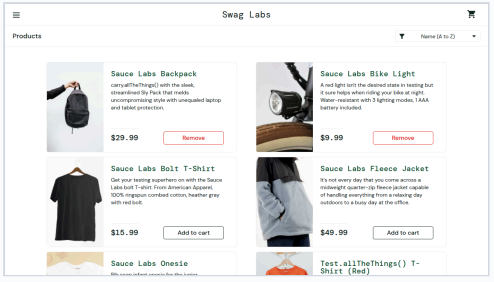
Actual	(a) The overview loads and the order completes with no customer data. `/checkout-complete.html` is also directly reachable. (b) Whitespace-only values are accepted and the order can be finished.
Impact	Orders with no usable name or address can't be shipped, which means support cost and refunds. The required-field check already exists but is trivially defeated.
Why a defect	The app clearly *intends* these fields to be required (it shows "First Name is required"). Accepting whitespace and allowing the step to be skipped undermine that rule. Format checks such as a numeric postal code were deliberately **not** reported, because no locale rule is stated.
Evidence	 ,  , 

D3 – Previous user's cart is shown to the next user who logs in on the same browser

Severity / Priority	Medium / P2
Steps	1. Log in as `standard_user`, add *Sauce Labs Fleece Jacket*. 2. Menu → **Logout**. 3. Log in as a different user (e.g. `visual_user`). 4. Open the cart.
Expected	Logout clears the session's cart, or the cart is scoped to the user. The second user sees an empty cart.
Actual	The second user's badge shows **1** and the cart contains the first user's *Fleece Jacket*. `localStorage["cart-contents"]` survives logout unchanged.

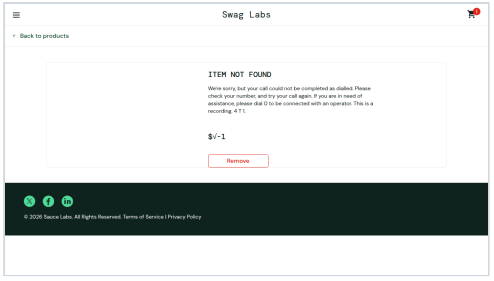
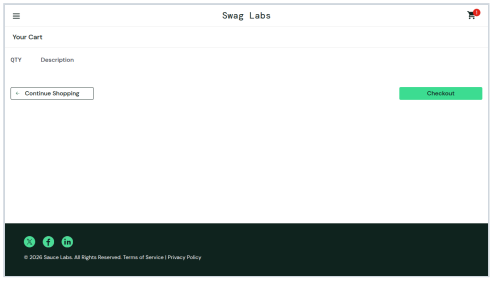
Impact	Privacy leak on shared or kiosk devices (it reveals what the previous person intended to buy). The next user could also buy items they never chose.
Why a defect	A cart that persists across a refresh for the *same* user is a reasonable feature. Carrying it across *a logout and a different identity* is not. It is not a seeded bug, because it reproduces starting from `standard_user`, and `visual_user` is only used as the second identity.
Evidence	

D4 – "Reset App State" empties the cart but the product list still shows "Remove"

Severity / Priority	Medium / P3
Steps	1. Log in, add *Backpack* and *Bike Light* (buttons change to **Remove**). 2. Menu → **Reset App State** → close menu.
Expected	Cart badge disappears **and** all buttons revert to **Add to cart**.
Actual	The badge disappears, but both products still show **Remove** until the page is reloaded. Clicking a stale **Remove** just flips it to **Add to cart**, so the UI and the cart disagree.
Impact	Users can't trust what is in their cart. It also matters for testers, who use Reset App State to clean state, and automated tests could produce false results.
Evidence	 (badge gone, two "Remove" buttons)

D5 – Non-existent product can be added to the cart, which then crashes the checkout overview (blank page)

Severity / Priority	Medium / P3. Low likelihood (needs URL tampering) but high consequence
Steps	1. Log in. 2. Open `https://www.saucedemo.com/inventory-item.html?id=999`. 3. Click **Add to cart**. 4. Open the cart, then **Checkout** → fill info → **Continue**.
Expected	Invalid products cannot be added (no button, or a server-side check), and the cart never stores unknown IDs.

Actual	Badge shows **1**, but the cart page lists **no items** (`cart-contents = [999]`). Continuing to the overview renders a **completely blank white page**, with no header, navigation or error.
Impact	The checkout is unusable, and the user is left with a phantom badge they can't remove from the cart page. It also shows that cart contents are never validated against the catalogue.
Why a defect	The "ITEM NOT FOUND / $√-1" page itself is a deliberate Sauce Labs joke, so I **don't** report it. Letting that placeholder be bought, and then crashing checkout, is a real robustness bug. I didn't check whether there is a way to recover.
Evidence	 ,  ,

4. Automation (Task C)

Repo: github.com/stern-batch/souceDemoAutomation (see `README.md` for setup and run instructions)

Flow automated: `standard_user` login → add *Backpack* + *Fleece Jacket* → check badge = 2 → check the cart has exactly those names, prices and quantity 1 → checkout → check the overview has the same items → item total equals the sum of the prices → tax is 8 % → total equals item total plus tax → Finish → confirmation page → badge gone, no "Remove" buttons, cart empty.

Negative tests: `locked_out_user` gets the exact error and no inventory. Checkout is blocked with the correct error when First Name, Last Name or Postal Code is missing (3 data-driven cases).

Result: 5/5 pass on Google Chrome (headless, about 3 seconds). With `ALL_BROWSERS=1`, 10/10 pass on Chrome and Firefox. **Mutation check:** I changed the expected tax rate to 9 % and the run failed

with `tax = 9% of item total — Expected 720, Received 640`, so the assertions are not vacuous.

Beyond the required flow: every test case in `docs/TEST_CASES.md` is automated, in three suites that run separately so known bugs can never turn the main run red.

Suite	Command	Covers	Result	In CI
Core	`npm test`	TC-01, 11, 13–15	✅ 5/5	Blocking
Regression	`npm run test:regression`	TC-02, 03, 04, 06, 07, 09, 12	✅ 18/18	Blocking
Known defects	`npm run test:defects`	TC-05, 08, 10, 16, 17, 18 (D1–D5)	❌ 6/6 fail	Informational (`continue-on-error`)

The known-defect tests assert the **correct** behaviour, so they are red while each bug exists. I deliberately did **not** use `test.fail()`, because that would show a green tick for a broken feature. Each failure message names the problem, for example `no product may still show "Remove": Expected 0, Received 2`. I checked every failure to confirm it comes from the defect itself, not from a timeout or a wrong selector. When a fix lands, the test turns green and moves into the regression suite.

What the AI generated

The page-object scaffold, fixtures, config, CI workflow, README draft, and the first versions of the specs and exploration probes.

What was wrong, brittle or unreliable, and what I changed

#	Problem in the AI output	Fix and reason
C1	Probes used `locator.textContent().catch(() => null)` to mean "element absent". `textContent()` auto-waits, so each missing element cost the full 30 s timeout, and 2 probes timed out.	Check `count()` first; in the suite, use `toHaveCount(0)` / `toBeHidden()`.
C2	A **race condition** that gave a false reading. The first probe read cart rows right after clicking the cart link and reported "6 items" for an empty cart, because it was still counting the inventory page. The overview step in the first spec had the same pattern.	Assert the URL, title and `toHaveCount(n)` before reading rows. This is why every page object has `expectLoaded()`.
C3	Screenshot taken while the slide-out menu was still animating (D4 evidence was partly covered).	Wait with `expect(menuLink).toBeHidden()` before the screenshot.
C4	In the first, TypeScript version of the suite, an untyped array failed the strict compiler check.	When I moved the suite to JavaScript, I kept the types as JSDoc comments (for example `LineItem`) with `// @ts-check`, so the editor still flags this kind of mistake. `npm run lint` syntax-checks every file in CI.

#	Problem in the AI output	Fix and reason
C5	The first probe used `Remove` buttons alone as the D4 oracle. Clicking a stale button "worked", which could have looked like a pass.	I also recorded the badge state and the button state after each click, which showed the UI and the cart disagree.
C6	Moving from TypeScript to JavaScript and from bundled Chromium to branded Chrome risked silent changes.	I re-ran the whole suite and every probe on Chrome 146, confirmed through `navigator.userAgentData` that the browser reports itself as "Google Chrome", and repeated the deliberate tax-rate failure check. I also caught that two probes would have overwritten the D3 and D4 evidence files with each other's screenshots after the earlier renumbering, and fixed their output paths.

The prompt asked for these design choices up front, and I checked each one in review rather than taking it on trust:

- **Selectors:** `getByTestId` (`data-test`) and `getByRole` , with cards located by visible product name and never by index.
- **Prices:** read from the catalogue at runtime instead of hard-coded.
- **Money maths:** in integer cents (`utils/money.js`) to avoid float drift.
- **Final assertions:** check the exact title, header and URL, then that the cart badge is gone, there are zero "Remove" buttons, and the cart is empty, so a test that stops early cannot pass.

Limitations and risks

- **External dependency:** tests hit the live saucedemo.com, so a site outage or content change fails CI. In a real project I'd run against a pinned environment.
- **UI login in every test** (about 1 s each). That's fine at this size. A larger suite should reuse `storageState` .
- **Business rules are encoded by observation:** the 8 % tax rate and the exact error copy were confirmed in the app, not taken from a specification.
- **Known-defect tests are red by design:** they are kept out of the blocking gate. On a real project, each one would also link to its bug ticket, so the team can see which red tests are tracked and which are new.
- **Coverage:** Google Chrome desktop by default, with Firefox opt-in. WebKit (Safari), Edge and mobile viewports are not covered. Chrome runs need branded Chrome installed (CI installs it with `npx playwright install chrome`).
- **No compile-time type checking:** JavaScript means type mistakes surface at runtime. JSDoc with `// @ts-check` gives editor warnings, but CI doesn't enforce them.

---

5. AI-driven QA strategy (Task D)

Where AI adds value

- **Requirements and design:** spotting ambiguities, deriving acceptance criteria, risk-based test ideas, boundary/equivalence tables.

- **Build:** drafting page objects, test data and test code; locator suggestions; turning bug reports into regression tests.
- **Execution:** clustering failures, summarising traces and logs, distinguishing flaky failures from real ones.
- **Maintenance:** proposing locator fixes and finding duplicate or dead tests.

Human review is mandatory for

- Deciding what "correct" means: oracles, expected values, business rules.
- Defect severity and go/no-go calls.
- Anything touching security, privacy, payments, or compliance.
- Deleting or "healing" a failing test. A green build must never be bought by weakening an assertion.

Review gate before an AI-generated test enters regression (PR checklist):

1. It traces to a requirement or risk and has no duplicates.
2. The assertions check the business outcome rather than just "element is visible", and the expected values come from a spec, not from whatever the app currently does.
3. It is proven to fail: the reviewer breaks the feature or mutates expected data and sees a red run.
4. It uses stable selectors (`data-test` or role), has no fixed sleeps, and owns its data and state.
5. It passes 10 repeated runs in CI without flakiness.
6. It contains no secrets or production data.
7. A named human reviewer approves it.

AI for analysis and optimisation

- **Defect analysis:** group failures by error signature or stack trace, link them to recent commits, and draft reproduction steps for a human to confirm.
- **Coverage:** map the code diff to requirements and existing tests to flag untested changed paths, as a *gap list* and not as a coverage percentage.
- **Regression optimisation:** risk-rank and select tests by the files changed and by failure history. The full suite still runs nightly, so the selector can't silently drop coverage.

CI/CD quality gates

Stage	Gate
Pull request	lint/type-check → unit tests → AI-selected impacted E2E tests plus the smoke suite (blocking) → AI review comments (advisory, never blocking on their own)
Merge to main	full E2E on 2 browsers
Nightly	full suite on all browsers, flaky-test report, AI failure triage summary
Release	human sign-off with the risk summary

Key risks and mitigations

Risk	Mitigation
Hallucinated assertions (AI asserts what the code *does*, not what it *should* do)	Expected values must come from specs or tickets, checked in review
False greens	Mutation or "make it fail" check at review, and ban `catch(() => {})` and `force: true`
Over-fitting to implementation	Assert user-visible outcomes and keep tests black-box
Data leakage (production data or credentials in prompts)	Approved enterprise AI tools only, synthetic data, secrets scanning in CI
Flaky tests	Quarantine lane, repeated-run gate, retries that report as flaky rather than hiding it
Automation bias (trusting AI because it sounds confident)	A named human owner per test and defect, and a periodic audit of AI-closed items

6. Tools used

Category	Tool
AI	Cursor (agent mode with a Claude model): exploration planning, probe scripts, test code, review, drafting this document
Automation	Playwright 1.63 (`@playwright/test`), JavaScript (CommonJS, JSDoc + `// @ts-check`), Node.js 22
Browsers	**Google Chrome 146** (primary, `channel: 'chrome'`), Firefox 155 (opt-in), mobile emulation 375×667
Inspection	Playwright `page.evaluate` for DOM, a11y attributes, `localStorage` and cookies; screenshots
CI	GitHub Actions (`.github/workflows/playwright.yml`), HTML report artifact

7. Evidence index

File	Defect
`evidence/D1a-empty-cart-overview.png`	D1: overview with $0.00 total

File	Defect
`evidence/D1b-empty-cart-order-complete.png`	D1: "Thank you for your order!" for an empty cart
`evidence/D2a-deeplink-overview-no-info.png`	D2: overview reached by URL with no customer info
`evidence/D2b-whitespace-only-info.png`	D2: whitespace-only values entered
`evidence/D2c-whitespace-accepted-overview.png`	D2: whitespace values accepted, Finish available
`evidence/D3-cart-carried-to-different-user.png`	D3: second user sees first user's Fleece Jacket
`evidence/D4-reset-state-stale-remove-buttons.png`	D4: badge cleared, two stale "Remove" buttons
`evidence/D5a-item-not-found-add-to-cart.png`	D5: placeholder item added, badge = 1
`evidence/D5b-cart-after-invalid-item.png`	D5: badge 1, cart empty
`evidence/D5c-checkout-overview-crash.png`	D5: blank checkout overview
`explore/*.probe.js`	Re-runnable probes behind every hypothesis (`npm run explore`)